

Universidad Autónoma de Nuevo León

Facultad de Ingeniería Mecánica y Eléctrica



Unidad de aprendizaje:

Control Óptimo

Producto Integrador de Aprendizaje (PIA)

Diseño e implementación de un control óptimo

que se presenta como parte de la

Actividad fundamental 3

Docente:

M. C. Miguel Cabrera Quiroz

Presenta:

González Alvarez Gabriel

Matrícula: 2109326

Grupo: 003

Días: Lunes, Miércoles y Viernes

Hora: N5

Aula: 3108

Fecha de entrega final: 29 de mayo de 2026

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León.

Índice

1. Introducción	2
2. Planteamiento del problema de control	3
3. Modelo matemático del sistema	3
3.1. Representación en variables de estado	4
3.2. Definición de matrices del sistema	4
4. Diseño del controlador óptimo LQR	5
4.1. Índice de desempeño	5
4.2. Matrices de ponderación	5
4.3. Ecuación algebraica de Riccati	5
4.4. Ganancia óptima de realimentación	6
5. Seguimiento de referencia mediante cambio de coordenadas	6
6. Implementación experimental en microcontrolador	7
6.1. Montaje experimental	8
7. Validación experimental y verificación de optimalidad	10
7.1. Adquisición de datos experimentales	10
7.2. Resultados experimentales	11
7.3. Video demostrativo	11
7.4. Cálculo del índice de desempeño	12
7.5. Comparación de controladores	12
7.6. Discusión de resultados	13
8. Conclusión	13
Apéndices	15
A. Códigos	15
A.1. Resolución numérica del controlador LQR	15
A.2. Análisis simbólico de la ecuación de Riccati	15

A.3. Resolución numérica del controlador LQI	16
A.4. Implementación en microcontrolador (ESP32)	17
A.5. Monitor de adquisición y evaluación experimental	24
B. Controladores	37
B.1. Control por asignación de polos	37
B.2. Control integral por asignación de polos	39
B.3. Observador de Luenberger	41

1. Introducción

El control moderno basado en espacio de estados constituye una de las herramientas más importantes para el análisis y diseño de sistemas dinámicos multivariantes. A diferencia de los métodos clásicos fundamentados únicamente en la función de transferencia, este enfoque permite incorporar de manera directa las variables de estado del sistema, facilitando el análisis de estabilidad, controlabilidad, observabilidad y desempeño dinámico.

Entre las técnicas más representativas del control moderno se encuentran la asignación de polos y el Regulador Cuadrático Lineal (LQR, por sus siglas en inglés). La asignación de polos permite ubicar los polos del sistema en posiciones deseadas para obtener determinadas características dinámicas, mientras que el controlador LQR determina automáticamente una ley de control óptima a partir de la minimización de un índice cuadrático de desempeño. Asimismo, la incorporación de acción integral da lugar a estrategias como el control integral por asignación de polos y el controlador LQI, los cuales permiten reducir el error en estado estacionario ante referencias constantes.

En este trabajo se desarrolla e implementa experimentalmente un sistema de control para una planta RC de segundo orden utilizando cuatro estrategias de control: asignación de polos, control integral por asignación de polos, LQR y LQI. El sistema fue implementado mediante un microcontrolador ESP32 programado en Arduino IDE, mientras que la adquisición, visualización y evaluación experimental de datos se realizó mediante una interfaz desarrollada en Python.

A diferencia de un enfoque exclusivamente teórico o de simulación, el presente trabajo se centra en la validación experimental de los controladores en tiempo real, considerando aspectos prácticos como adquisición de señales analógicas, saturación del actuador, discretización temporal, comunicación serial y monitoreo gráfico de variables de estado, salida y señal de control.

Además del diseño matemático de los controladores, se desarrolló un monitor experimental capaz de visualizar el comportamiento dinámico de cada estrategia de control y calcular automáticamente índices de desempeño a partir de los datos adquiridos. Esto permitió realizar una comparación experimental directa entre los distintos métodos implementados.

Finalmente, se presentan los resultados experimentales obtenidos para cada controlador, analizando su comportamiento dinámico, capacidad de seguimiento y desempeño general dentro de la planta física implementada.

2. Planteamiento del problema de control

Una vez obtenido el modelo dinámico de la planta RC de segundo orden en representación de espacio de estados, el problema de control consiste en diseñar leyes de control capaces de regular el comportamiento del sistema y garantizar un seguimiento adecuado de referencias de voltaje.

El sistema implementado presenta dinámica de segundo orden y cuenta con dos variables de estado asociadas a los voltajes en los capacitores de la red RC. Debido a ello, resulta posible aplicar técnicas de control moderno basadas en retroalimentación de estados.

En este trabajo se consideran cuatro estrategias de control:

- ✓ Control por asignación de polos.
- ✓ Control integral por asignación de polos.
- ✓ Regulador cuadrático lineal (LQR).
- ✓ Regulador cuadrático lineal con acción integral (LQI).

El objetivo principal es comparar experimentalmente el desempeño dinámico de cada estrategia de control en términos de seguimiento de referencia, rapidez de respuesta, error en estado estacionario y esfuerzo de control.

Para ello, los controladores fueron implementados en tiempo real mediante un microcontrolador ESP32, mientras que la adquisición y visualización de datos experimentales se realizó utilizando una interfaz desarrollada en Python. Las señales de salida y control obtenidas experimentalmente permitieron evaluar el comportamiento de cada controlador y calcular índices de desempeño asociados al costo cuadrático del sistema.

3. Modelo matemático del sistema

El sistema bajo estudio corresponde a una red eléctrica compuesta por resistencias y capacitores. En la Figura 1 se muestra el circuito equivalente.

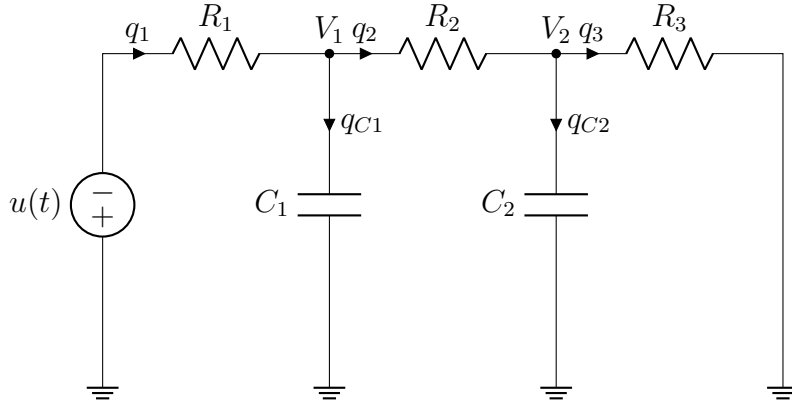


Figura 1: Circuito RC del sistema

Aplicando la Ley de Corrientes de Kirchhoff:

$$\frac{u - V_1}{R_1} = C_1 \frac{dV_1}{dt} + \frac{V_1 - V_2}{R_2} \quad (1)$$

$$\frac{V_1 - V_2}{R_2} = C_2 \frac{dV_2}{dt} + \frac{V_2}{R_3} \quad (2)$$

3.1. Representación en variables de estado

Definiendo:

$$x_1 = V_1, \quad x_2 = V_2, \quad y = x_2 \quad (3)$$

Se obtienen las ecuaciones de estado:

$$\dot{x}_1 = \left(-\frac{1}{R_1 C_1} - \frac{1}{R_2 C_1} \right) x_1 + \frac{1}{R_2 C_1} x_2 + \frac{1}{R_1 C_1} u \quad (4)$$

$$\dot{x}_2 = \frac{1}{R_2 C_2} x_1 - \left(\frac{1}{R_2 C_2} + \frac{1}{R_3 C_2} \right) x_2 \quad (5)$$

3.2. Definición de matrices del sistema

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} -\frac{1}{R_1 C_1} - \frac{1}{R_2 C_1} & \frac{1}{R_2 C_1} \\ \frac{1}{R_2 C_2} & -\frac{1}{R_2 C_2} - \frac{1}{R_3 C_2} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} \frac{1}{R_1 C_1} \\ 0 \end{bmatrix} u \quad (6)$$

$$y = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad (7)$$

4. Diseño del controlador óptimo LQR

4.1. Índice de desempeño

Con el objetivo de diseñar un controlador óptimo, se define un índice de desempeño que permite evaluar la calidad de la respuesta del sistema. Este índice se plantea como:

$$J = \int_0^{\infty} \left(x^T(t)Qx(t) + u^T(t)Ru(t) \right) dt \quad (8)$$

donde Q es una matriz de ponderación de estados y R es una ponderación asociada al esfuerzo de control.

La formulación anterior corresponde al problema de regulación cuadrática lineal, abordado en textos clásicos de control óptimo y control moderno [1, 2].

4.2. Matrices de ponderación

Dado que se desea dar mayor importancia al seguimiento del voltaje en el capacitor 2, se establece que dicho estado tenga un peso mayor. Por lo tanto, la matriz Q se define como:

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 5 \end{bmatrix} \quad (9)$$

Por otro lado, R se selecciona como un escalar positivo que penaliza el uso excesivo de la señal de control:

$$R = 1 \quad (10)$$

Este planteamiento permite balancear el compromiso entre el desempeño del sistema y el esfuerzo requerido por el actuador.

4.3. Ecuación algebraica de Riccati

El problema de optimización se resolvió mediante la solución de la ecuación algebraica de Riccati asociada al problema de regulación cuadrática lineal descrito en [2], utilizando Python. El código completo se presenta en el Apéndice A.1. A continuación se muestran las líneas relevantes correspondientes al cálculo de la solución de Riccati y la ganancia LQR:

```
25 P = solve_continuous_are(A,B,Q,R)
```

Listing 1: Cálculo de la ganancia LQR

```
30 K = np.linalg.inv(R) @ B.T @ P
```

Listing 2: Cálculo de la ganancia LQR (continuación)

4.4. Ganancia óptima de realimentación

Se obtuvo:

$$K = \begin{bmatrix} 0.41421356 & 0.41421356 \end{bmatrix}$$

De manera adicional, se desarrollaron estrategias de control mediante asignación de polos, control integral por asignación de polos y control óptimo con acción integral (LQI), con el propósito de comparar experimentalmente su desempeño respecto al controlador LQR.

En el caso del controlador LQI, el sistema fue aumentado incorporando un estado integral asociado al error de seguimiento, resolviendo posteriormente la ecuación algebraica de Riccati mediante el mismo procedimiento utilizado para el LQR convencional.

Por otro lado, los controladores basados en asignación de polos emplean un observador de estados para estimar el voltaje asociado al primer capacitor, debido a que únicamente se mide directamente la salida del sistema. El desarrollo matemático del observador y de las estrategias de control implementadas se presenta en el Apéndice B.

Asimismo, el código correspondiente a la solución numérica del controlador LQI mediante Python se presenta en el Apéndice A.3.

5. Seguimiento de referencia mediante cambio de coordenadas

Para garantizar el seguimiento de una referencia constante sin error en estado estacionario, se emplea un enfoque basado en cambio de coordenadas, el cual permite transformar el problema de seguimiento en uno de regulación.

Bajo este enfoque, la ley de control se plantea como:

$$u = -Kx + gr \quad (11)$$

donde K es la ganancia de retroalimentación de estados y g es una ganancia de ajuste encargada de asegurar el valor deseado en estado estacionario.

$$g = \begin{bmatrix} K & 1 \end{bmatrix} \begin{bmatrix} A & B \\ C & D \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad (12)$$

donde $K = \begin{bmatrix} k_1 & k_2 \end{bmatrix}$ y las matrices corresponden al modelo definido en (6) y (7).

La matriz aumentada es:

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} = \begin{bmatrix} -\frac{1}{R_1 C_1} - \frac{1}{R_2 C_1} & \frac{1}{R_2 C_1} & \frac{1}{R_1 C_1} \\ \frac{1}{R_2 C_2} & -\frac{1}{R_2 C_2} - \frac{1}{R_3 C_2} & 0 \\ 0 & 1 & 0 \end{bmatrix} \quad (13)$$

Finalmente, al realizar las operaciones correspondientes, se obtiene:

$$g = k_1 + k_2 + \frac{k_1 R_2 + R_1 + R_2}{R_3} + 1 \quad (14)$$

Esta expresión se utilizó para calcular la ganancia de prealimentación del controlador LQR, cuyo código completo se presenta en el Apéndice A.4, correspondiente a la implementación en el microcontrolador. En particular, la instrucción correspondiente se muestra a continuación:

152

```
Kp_3 = K1_3 + K2_3 + 1 + (K1_3*R2+R1+R2)/R3;  //Ganancia de seguimiento mediante
cambio de coordenadas
```

Listing 3: Cálculo de la ganancia de seguimiento para el controlador LQR

Esta ganancia permite garantizar que la salida del sistema siga la referencia deseada sin error en estado estacionario, siempre que el modelo represente adecuadamente la dinámica del sistema.

La estrategia de seguimiento mediante prealimentación y cambio de coordenadas empleada en este trabajo corresponde al enfoque clásico presentado en [1]. La implementación práctica utilizada para el cálculo de la ganancia de seguimiento y su adaptación al sistema RC experimental fue desarrollada tomando como referencia la metodología y ejemplos disponibles en [4].

6. Implementación experimental en microcontrolador

La ley de control implementada es:

$$u = -Kx + gr$$

El fragmento principal del código correspondiente a esta ley se muestra en el Código 4, mientras que el programa completo del microcontrolador se presenta en el Apéndice A.4.

El programa completo implementado en el ESP32 incluye las cuatro estrategias de control evaluadas experimentalmente: asignación de polos, control integral por asignación de polos, control óptimo LQR y control con acción integral (LQI). Parte de la estructura de programación utilizada para la implementación de los controladores en el microcontrolador, así como la organización general del sistema experimental, se desarrolló tomando como referencia los ejemplos y prácticas disponibles en [4].

```
225 // ===== Control LQR =====
226 void control_3(){
227     X13 = analogRead(X1_3)*mX; //estado 1
228     X23 = analogRead(X2_3)*mX; //estado 2
229     Y3 = X23; //salida
230     //Ley de control u = -Kx
231     U3 = Kp_3*R - K1_3*X13 - K2_3*X23;
232
233     U3 = constrain(U3,0,3.3); //saturacion
234 }
```

Listing 4: Control LQR implementado

Donde x_1 y x_2 corresponden a los voltajes medidos en cada capacitor del sistema.

En las estrategias basadas en asignación de polos se empleó un observador de estados de orden completo para estimar el voltaje correspondiente al estado x_1 , utilizando únicamente la medición directa de la salida del sistema.

Por otro lado, en las implementaciones LQR y LQI se realizó la medición directa de ambos estados, por lo que no fue necesario utilizar estimación mediante observador. El desarrollo y código completo del observador implementado se presentan en el Apéndice B.3.

6.1. Montaje experimental

Con el objetivo de validar experimentalmente el modelo desarrollado y comparar distintas estrategias de control bajo condiciones equivalentes, se realizó la implementación física del sistema utilizando una tarjeta de pruebas (protoboard) y un microcontrolador ESP32.

El circuito RC descrito previamente en la Figura 1 fue replicado en cuatro ocasiones dentro de la misma tarjeta, empleando valores idénticos de resistencias y capacitores para cada configuración. Esta disposición permite ejecutar múltiples estrategias de control de manera simultánea sobre sistemas físicamente equivalentes.

La implementación digital de las conexiones se muestra en la Figura 2. En este diagrama se presentan las conexiones generales entre las plantas RC, el microcontrolador ESP32, las señales de adquisición y las salidas PWM utilizadas para cada controlador. Con el propósito de mejorar la legibilidad del diagrama, se emplearon etiquetas de conexión en lugar de rutas físicas completas de cableado.

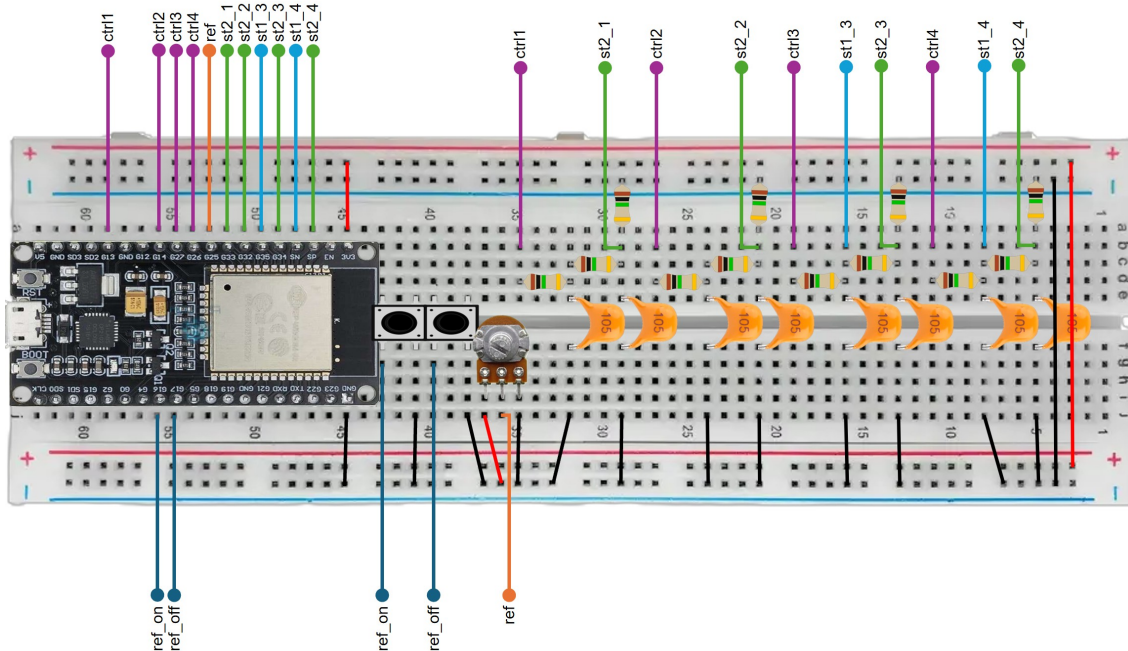


Figura 2: Diagrama general de conexiones del sistema implementado en protoboard.

Por otro lado, la implementación física final del sistema se muestra en la Figura 3, donde se observan las cuatro configuraciones RC construidas en paralelo sobre la tarjeta de pruebas.

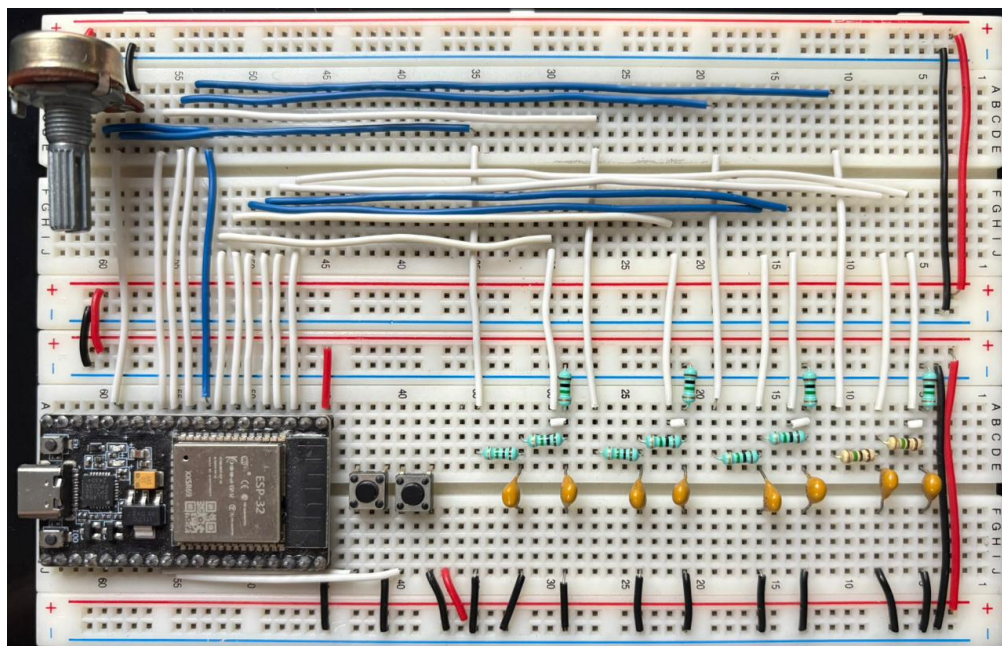


Figura 3: Implementación física del sistema RC en protoboard.

La arquitectura implementada permite comparar simultáneamente distintas estrategias de control utilizando la misma referencia, el mismo tiempo de muestreo y condiciones fí-

sicas equivalentes. De esta manera, es posible evaluar el desempeño de cada controlador considerando variables como el seguimiento de referencia, el esfuerzo de control y la respuesta dinámica del sistema. Cada planta fue asociada a una estrategia de control distinta, permitiendo realizar la comparación experimental de manera simultánea y bajo condiciones equivalentes.

Adicionalmente, el ESP32 se encarga de realizar la adquisición de señales analógicas, el cálculo de las leyes de control en tiempo discreto y la generación de señales PWM aplicadas a cada una de las plantas implementadas.

7. Validación experimental y verificación de optimalidad

Con el objetivo de verificar experimentalmente la optimalidad del controlador diseñado respecto al índice de desempeño definido en la Sección 4.1, se realizó la comparación entre el control óptimo LQR y tres estrategias de control adicionales: control por asignación de polos, control integral por asignación de polos y control con acción integral (LQI).

El controlador LQR se considera óptimo respecto al índice de desempeño planteado, debido a que su ganancia de retroalimentación se obtiene mediante la minimización explícita de dicha función de costo.

De manera similar, el controlador LQI también corresponde a una estrategia de control óptimo; sin embargo, su diseño se realiza sobre un sistema aumentado que incorpora un estado integral, por lo que minimiza una función de costo distinta asociada al sistema extendido.

Por otro lado, los controladores basados en asignación de polos no optimizan explícitamente una función de costo, sino que se diseñan a partir de especificaciones dinámicas deseadas.

Las mediciones fueron adquiridas mediante un sistema de monitoreo desarrollado en Python, el cual permite la visualización en tiempo real de las trayectorias de referencia, salida y señal de control de cada uno de los sistemas implementados. El sistema también almacena automáticamente los datos experimentales en archivos CSV para su posterior análisis.

7.1. Adquisición de datos experimentales

La adquisición de datos se realizó mediante comunicación serial entre el microcontrolador ESP32 y una interfaz desarrollada con *PySide6*, mientras que la visualización se implementó utilizando *PyQtGraph*. Este sistema permite la recepción de paquetes binarios que contienen las variables del sistema, los cuales son decodificados y almacenados en estructuras tipo cola para su visualización en tiempo real. La arquitectura general del sistema de monitoreo y adquisición de datos fue desarrollada tomando como referencia las herramientas y ejemplos disponibles en [4], realizando adaptaciones específicas para la adquisición simultánea de múltiples plantas y el cálculo experimental del índice de desempeño.

Al finalizar cada experimento, los datos son exportados a formato CSV mediante la librería *pandas*, lo que permite su análisis numérico y la evaluación del desempeño de cada controlador.

7.2. Resultados experimentales

En la Figura 4 se muestra un ejemplo del reporte generado automáticamente por el sistema de monitoreo, donde se observan las respuestas de los cuatro controladores ante la misma referencia.

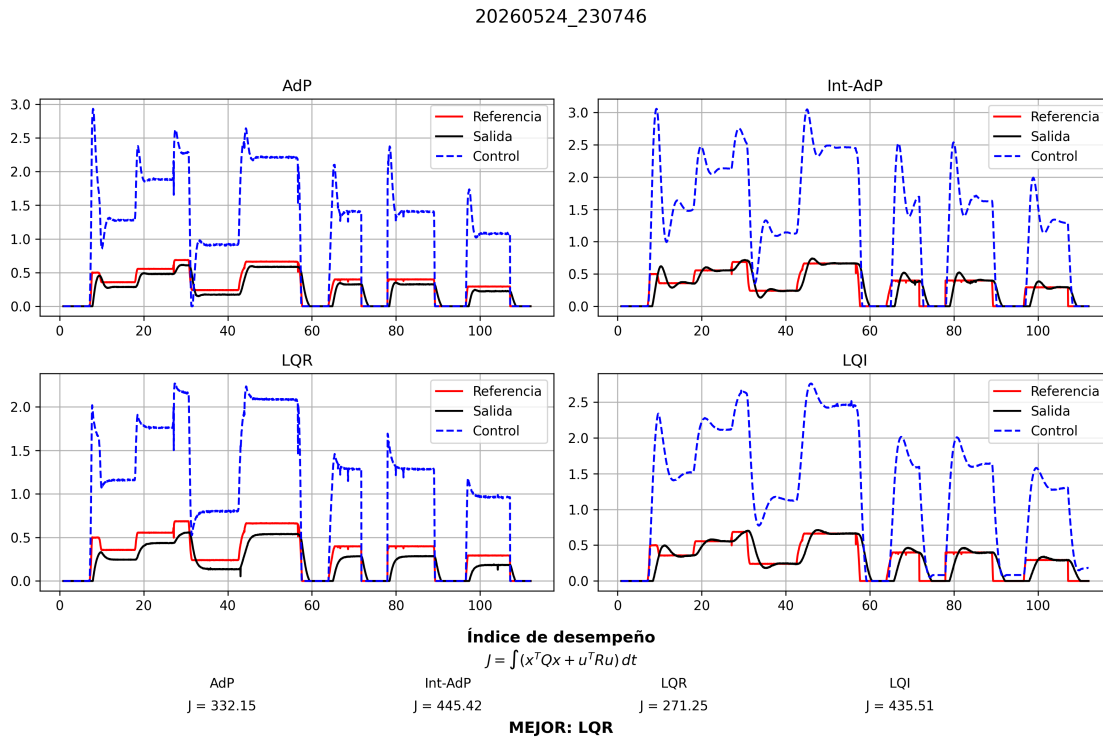


Figura 4: Resultados experimentales obtenidos mediante el sistema de monitoreo en tiempo real.

A partir de los datos obtenidos, se evaluó el índice de desempeño definido previamente. Este cálculo se realizó de forma numérica utilizando la aproximación discreta del criterio cuadrático, considerando las trayectorias de los estados y la señal de control registrada durante el experimento.

7.3. Video demostrativo

Con el propósito de complementar la validación experimental del sistema, se realizó un video demostrativo donde se presenta el funcionamiento general de la implementación física, así como la comparación entre las distintas estrategias de control utilizadas.

El video puede consultarse en el siguiente enlace:

<https://youtu.be/9gk9sejwvD8>

7.4. Cálculo del índice de desempeño

El índice de desempeño fue implementado directamente en el sistema de monitoreo, utilizando los datos almacenados en el archivo CSV generado al finalizar la adquisición.

El fragmento principal del código utilizado para este cálculo se muestra a continuación:

```
247 def calcular_J(self, df):
248
249     dt = df['t'].diff().fillna(0)
250
251     resultados = []
252
253     for i in range(1,5):
254         x1 = df[f'X1_{i}']
255         x2 = df[f'Y_{i}']
256         u = df[f'U_{i}']
257
258         J = ((x1**2 + 5*x2**2 + u**2) * dt).sum()
259         resultados.append(J)
260
261     nombres = ["AdP", "Int-AdP", "LQR", "LQI"]
262
263     return resultados, nombres
```

Listing 5: Cálculo del índice de desempeño a partir de datos experimentales

Como se observa, el índice de desempeño se calcula de forma independiente para cada controlador, considerando la contribución de los estados y la energía de control.

7.5. Comparación de controladores

Los resultados obtenidos permiten comparar el desempeño de los cuatro esquemas de control bajo las mismas condiciones experimentales. El sistema de monitoreo genera automáticamente una comparación numérica del índice de desempeño J , identificando el controlador con menor valor como el de mejor desempeño respecto al criterio planteado.

Como puede observarse en la Figura 4, el controlador LQR presenta el menor valor de J , lo cual resulta consistente con la teoría de control óptimo, ya que su ganancia fue obtenida mediante la minimización explícita del funcional definido en la Sección 4.1.

Por otro lado, aunque el controlador LQI también corresponde a una estrategia de control óptimo, este minimiza una función de costo distinta asociada al sistema aumentado con acción

integral. Debido a ello, el desempeño obtenido respecto al índice original resulta inferior al obtenido mediante el LQR convencional.

Asimismo, los controladores basados en asignación de polos presentan valores mayores de J , debido a que su diseño se basa en especificaciones dinámicas deseadas y no en la minimización explícita de una función de costo cuadrática.

7.6. Discusión de resultados

Los resultados experimentales validan que el controlador LQR presenta el mejor desempeño global respecto al índice de desempeño definido, obteniendo el menor valor de J entre todas las estrategias implementadas.

Aunque el controlador LQI incorpora acción integral y corresponde también a una estrategia de control óptimo, su diseño se realiza sobre un sistema aumentado y con una función de costo distinta, lo que produjo un incremento considerable en el valor del índice evaluado experimentalmente.

Por otro lado, los controladores diseñados mediante asignación de polos presentaron desempeños intermedios y mayores valores de esfuerzo de control, debido a que no consideran explícitamente un criterio de optimización durante su diseño.

En conjunto, los resultados experimentales confirman que la formulación del problema de control como una minimización explícita de un índice cuadrático permite obtener un mejor compromiso entre seguimiento de referencia y esfuerzo de control.

8. Conclusión

En este trabajo se obtuvo el modelo matemático de un sistema eléctrico RC en forma de espacio de estados a partir del análisis del circuito mediante las leyes de Kirchhoff. Este modelo permitió describir la dinámica del sistema en función de los voltajes presentes en los capacitores y sirvió como base para el diseño de distintas estrategias de control.

Asimismo, se definió un índice de desempeño cuadrático que permitió formular el problema de control óptimo y diseñar un controlador LQR mediante la solución de la ecuación algebraica de Riccati. De manera complementaria, se desarrollaron estrategias adicionales de control mediante asignación de polos, control integral por asignación de polos y control óptimo con acción integral (LQI), con el propósito de comparar experimentalmente su desempeño.

Para las estrategias basadas en asignación de polos se implementó un observador de estados de orden completo, permitiendo estimar el estado no medido del sistema a partir de la salida disponible. Por otro lado, las implementaciones LQR y LQI emplearon medición directa de ambos estados.

Adicionalmente, se determinó una ganancia de prealimentación mediante cambio de coordenadas, permitiendo garantizar el seguimiento de referencias constantes sin error en

estado estacionario para el controlador LQR.

Finalmente, se realizó la implementación experimental de cuatro plantas RC físicamente equivalentes utilizando un microcontrolador ESP32, lo que permitió ejecutar simultáneamente las distintas estrategias de control bajo las mismas condiciones experimentales. Asimismo, se desarrolló un monitor de adquisición y evaluación experimental en Python para la visualización en tiempo real de las variables del sistema y la generación automática de métricas de desempeño.

Los resultados obtenidos validan que el controlador LQR presenta el mejor desempeño respecto al índice de desempeño planteado, confirmando experimentalmente que la minimización explícita del funcional definido conduce a un mejor compromiso entre seguimiento de referencia y esfuerzo de control para el sistema considerado.

En conjunto, el trabajo integra modelado matemático, diseño de controladores, estimación de estados, programación embebida, adquisición de datos y validación experimental, proporcionando una implementación completa de técnicas de control moderno aplicadas a un sistema dinámico real.

Con el propósito de facilitar la reproducción y continuidad del trabajo desarrollado, el proyecto y los archivos complementarios utilizados durante la implementación se encuentran disponibles en el siguiente repositorio:

<https://github.com/gabrielgonal-hub/Control-Optimo>

Referencias

- [1] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [2] D. E. Kirk, *Optimal Control Theory: An Introduction*, Mineola, NY, USA: Dover Publications, 2004.
- [3] G. F. Franklin, J. D. Powell, and A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 7th ed. Boston, MA, USA: Pearson, 2015.
- [4] M. I. Platas Garza, *lab-control-fime*, GitHub repository, Disponible en: <https://github.com/miplatas/lab-control-fime> [Último acceso: mayo 2026].

Apéndices

A. Códigos

A.1. Resolución numérica del controlador LQR

```
1 import numpy as np
2 from scipy.linalg import solve_continuous_are
3 from scipy.integrate import solve_ivp
4 import matplotlib.pyplot as plt
5
6 R1 = 1000
7 R2 = 1000
8 R3 = 1000
9 C1 = 1e-6
10 C2 = 1e-6
11
12 # matrices del sistema
13 A = np.array([[ -1/(R1*C1)-1/(R2*C1), 1/(R2*C1)],
14               [ 1/(R2*C2), -1/(R2*C2)-1/(R3*C2)]])
15
16 B = np.array([[1/(R1*C1)],
17               [0]])
18
19 Q = np.array([[1, 0],
20               [0, 5]])
21
22 R = np.array([[1]])
23
24 # resolver Riccati
25 P = solve_continuous_are(A,B,Q,R)
26
27 print("P =", P)
28
29 # ganancia LQR
30 K = np.linalg.inv(R) @ B.T @ P
31 print("K =", K)
```

A.2. Análisis simbólico de la ecuación de Riccati

```
1 import numpy as np
2 from scipy.linalg import solve_continuous_are
3 from sympy import symbols, Matrix, solve, pprint
4
5 R1, C1, R2, C2, R3 = symbols('R1 C1 R2 C2 R3')
```

```

6
7 A = Matrix([[ -1/(R1*C1)-1/(R2*C1), 1/(R2*C1)],
8           [1/(R2*C2), -1/(R2*C2)-1/(R3*C2)]])
9
10 B = Matrix([[1/(R1*C1)],
11            [0]])
12
13 Q = Matrix([[1, 0],
14            [0, 5]])
15
16 R = Matrix([1])
17
18 p11, p12, p22 = symbols('p11 p12 p22')
19
20 P = Matrix([[p11, p12],
21            [p12, p22]])
22
23 Rinv = R.inv()
24
25 riccati = A.T*P + P*A - P*B*Rinv*B.T*P + Q
26
27 pprint(riccati)

```

A.3. Resolución numérica del controlador LQI

El controlador LQI fue desarrollado mediante el aumento del sistema con acción integral y la posterior solución de la ecuación algebraica de Riccati, siguiendo el enfoque presentado en [2].

```

1 #Control LQI
2 import numpy as np
3 from scipy.linalg import solve_continuous_are
4 from scipy.integrate import solve_ivp
5 import matplotlib.pyplot as plt
6
7 R1 = 1000
8 R2 = 1000
9 R3 = 1000
10 C1 = 1e-6
11 C2 = 1e-6
12
13 # matrices del sistema
14 A = np.array([[ -1/(R1*C1)-1/(R2*C2), 1/(R2*C1)],
15             [1/(R2*C2), -1/(R2*C2)-1/(R3*C2)]])
16
17 B = np.array([[1/(R1*C1)],
18             [0]])

```

```

19
20 C = np.array([[0, 1]])
21
22 # sistema aumentado (LQI)
23
24 Aa = np.block([
25     [np.zeros((1,1)), -C],
26     [np.zeros((2,1)), A]
27 ])
28
29 Ba = np.vstack([
30     np.zeros((1,1)),
31     B
32 ])
33
34 print("Aa =\n", Aa)
35 print("Ba =\n", Ba)
36
37 Qa = np.diag([10, 1, 5])
38
39 print("Qa =\n", Qa)
40
41 R = np.array([[1]])
42
43 # resolver Riccati
44 P = solve_continuous_are(Aa,Ba,Qa,R)
45
46 print("P =", P)
47
48 # ganancia LQI
49 K = np.linalg.inv(R) @ Ba.T @ P
50 print("K =", K)

```

Se obtuvo:

$$K = \begin{bmatrix} -3.16227766 & 0.41495869 & 0.41601274 \end{bmatrix}$$

A.4. Implementación en microcontrolador (ESP32)

```

1 // ===== PINES =====
2 #define X2_1 33
3 #define X2_2 32
4 #define X1_3 35
5 #define X2_3 34
6 #define X1_4 39
7 #define X2_4 36

```

```

8
9 #define REF_PIN 25
10
11 #define PWM1 13
12 #define PWM2 14
13 #define PWM3 27
14 #define PWM4 26
15
16 #define BTN_ON 16
17 #define BTN_OFF 17
18
19 // ===== PWM =====
20 #define PWM_FREQ 5000
21 #define PWM_RES 8
22
23 // ===== PLANTA =====
24 #define R1 1000000
25 #define R2 1000000
26 #define R3 1000000
27 #define C1 0.000001
28 #define C2 0.000001
29
30 #define mX (3.3/4095.0)
31 #define mU (255.0/3.3)
32
33 // ===== TIEMPO =====
34 unsigned long TS = 50;
35 float Tseg = 0.05;
36 unsigned long TIC = 0, TS_code = 0, TC = 0;
37
38 // ===== VARIABLES =====
39 float R = 0;
40 bool Habilitado = 0;
41
42 // ===== MODELO =====
43 float Am11, Am12, Am21, Am22, Bm1, Bm2;
44
45 // ===== SISTEMA 1 (POLOS) =====
46 float Xe1_1=0, Xe2_1=0;
47 float K1_1, K2_1, Kp_1;
48 float H1_1, H2_1;
49 float U1=0, Y1=0;
50
51 // ===== SISTEMA 2 (INTEGRAL POLOS) =====
52 float Xe1_2=0, Xe2_2=0, X0_2=0;
53 float K0_2, K1_2, K2_2;
54 float H1_2, H2_2;
55 float U2=0, Y2=0;

```

```
56
57 // ===== SISTEMA 3 (LQR) =====
58 float K1_3=0.41421356;
59 float K2_3=0.41421356;
60 float Kp_3;
61 float X13=0, X23=0;
62 float U3=0, Y3=0;
63
64 // ===== SISTEMA 4 (LQI) =====
65 float K0_4=3.16227766;
66 float K1_4=0.41495869;
67 float K2_4=0.41601274;
68 float X0_4=0;
69 float X14=0, X24=0;
70 float U4=0, Y4=0;
71
72 // =====
73 // ===== SETUP =====
74 // =====
75 void setup(){
76     Serial.begin(115200);
77
78     pinMode(BTN_ON, INPUT_PULLUP);
79     pinMode(BTN_OFF, INPUT_PULLUP);
80
81     // PWM
82     ledcSetup(0, PWM_FREQ, PWM_RES);
83     ledcSetup(1, PWM_FREQ, PWM_RES);
84     ledcSetup(2, PWM_FREQ, PWM_RES);
85     ledcSetup(3, PWM_FREQ, PWM_RES);
86
87     ledcAttachPin(PWM1,0);
88     ledcAttachPin(PWM2,1);
89     ledcAttachPin(PWM3,2);
90     ledcAttachPin(PWM4,3);
91
92     // ===== MODELO =====
93     float p11 = R1*C1;
94     float p12 = R1*C2;
95     float p21 = R2*C1;
96     float p22 = R2*C2;
97     float p32 = R3*C2;
98
99     Am11 = -1/p11 - 1/p21;
100    Am12 = 1/p21;
101    Am21 = 1/p22;
102    Am22 = -1/p22 - 1/p32;
103    Bm1 = 1/p11;
```

```

104 Bm2 = 0;
105
106 // =====
107 // SISTEMA 1 (ASIGNACIÓN DE POLOS)
108 // =====
109 float a1r=-1.25, a1i=-1.7320508; //polos controlador
110 float a2r=-1.25, a2i=1.7320508;
111 float b1r=-10, b2r=-10; //polos observador
112
113 float sum_a = -(a1r + a2r);
114 float prod_a = a1r*a2r - a1i*a2i;
115 float sum_b = -(b1r + b2r);
116 float prod_b = b1r*b2r;
117
118
119 //Ganancias de controlador
120 K1_1 = p11*(sum_a - 1/p11 - 1/p21 - 1/p22 - 1/p32);
121 K2_1 = p12*p22*(prod_a - 1/(p11*p22) - 1/(p11*p32) - 1/(p21*p22) - 1/(p21*p32)
122       - K1_1/(p11*p22) - K1_1/(p11*p32) + 1/(p21*p22));
123 Kp_1 = K1_1 + K2_1 + 1 + (K1_1*R2+R1+R2)/R3; //Ganancia de seguimiento mediante
       cambio de coordenadas
124
125
126 //Observador de Luenberger
127 H2_1 = sum_b - 1/p11 - 1/p21 - 1/p22 - 1/p32;
128 H1_1 = p22*(prod_b - 1/(p11*p22) - 1/(p11*p32) - 1/(p21*p22)
129       - 1/(p21*p32) - H2_1/p11 - H2_1/p21 + 1/(p21*p22));
130
131 // =====
132 // SISTEMA 2 (INTEGRAL)
133 // =====
134 float a3r=-1; //polo integral
135
136 float sum_ai = -(a1r + a2r + a3r);
137 float prod_ai = -(a1r*a2r - a1i*a2i)*a3r;
138
139 //Ganancias de controlador
140 K0_2 = p11*p22*prod_ai;
141 K1_2 = p11*(sum_ai - 1/p11 - 1/p21 - 1/p22 - 1/p32);
142 K2_2 = p11*(p22*(a1r*a3r + a2r*a3r + a1r*a2r)
143       - 1/(p21*p32) - (1+K1_2)/(p11*p32)) - 1 - K1_2;
144
145 //Mismo observador de Luenberger
146 H2_2 = H2_1;
147 H1_2 = H1_1;
148
149 // =====
150 // SISTEMA 3 (LQR)

```

```

151 // =====
152 Kp_3 = K1_3 + K2_3 + 1 + (K1_3*R2+R1+R2)/R3; //Ganancia de seguimiento mediante
      cambio de coordenadas
153
154 }
155
156 // =====
157 // ===== LOOP =====
158 // =====
159 void loop(){
160
161     if(digitalRead(BTN_ON) == LOW) Habilitado=1;
162     else if(digitalRead(BTN_OFF) == LOW) Habilitado=0;
163
164     R = Habilitado*(analogRead(REF_PIN)*mX); //Referencia
165
166     control_1();
167     control_2();
168     control_3();
169     control_4();
170
171     // ===== PWM =====
172     ledcWrite(0, U1*mU);
173     ledcWrite(1, U2*mU);
174     ledcWrite(2, U3*mU);
175     ledcWrite(3, U4*mU);
176
177     // coms_arduino_ide();
178
179     coms_python(&R,
180                 &Xe1_1,&Y1,&U1, //sistema 1
181                 &Xe1_2,&Y2,&U2, //sistema 2
182                 &X13,&Y3,&U3,   //sistema 3
183                 &X14,&Y4,&U4); //sistema 4
184
185     espera();
186 }
187
188 // ===== Control por Asignación de polos =====
189 void control_1(){
190     Y1 = analogRead(X2_1)*mX; //estado 2 y salida
191     float e1 = Y1 - Xe2_1;
192
193     float f1_1 = Am11*Xe1_1 + Am12*Xe2_1 + Bm1*U1 + H1_1*e1; //Dinamica estado 1
      observador
194     float f2_1 = Am21*Xe1_1 + Am22*Xe2_1 + H2_1*e1;           //Dinamica estado 2
      observador
195

```

```

196 Xe1_1 += Tseg*f1_1; //Integrador estado 1 mediante Euler
197 Xe2_1 += Tseg*f2_1; //Integrador estado 2 mediante Euler
198
199 //Ley de control u = -Kx
200 U1 = Kp_1*R - K1_1*Xe1_1 - K2_1*Xe2_1;
201
202 U1 = constrain(U1,0,3.3); //saturacion
203 }
204
205 // ===== Control Integral por Asignación de polos =====
206 void control_2(){
207     Y2 = analogRead(X2_2)*mX; //estado 2 y salida
208
209     //observador
210     float e2 = Y2 - Xe2_2;
211
212     float f1_2 = Am11*Xe1_2 + Am12*Xe2_2 + Bm1*U2 + H1_2*e2; //Dinámica estado 1
        observador
213     float f2_2 = Am21*Xe1_2 + Am22*Xe2_2 + H2_2*e2; //Dinámica estado 2
        observador
214
215     Xe1_2 += Tseg*f1_2; //Integrador estado 1 mediante Euler
216     Xe2_2 += Tseg*f2_2; //Integrador estado 2 mediante Euler
217     X0_2 += Tseg*(R - Y2); //estado 0
218
219     //Ley de control u = -Kx
220     U2 = K0_2*X0_2 - K1_2*Xe1_2 - K2_2*Xe2_2;
221
222     U2 = constrain(U2,0,3.3); //saturación
223 }
224
225 // ===== Control LQR =====
226 void control_3(){
227     X13 = analogRead(X1_3)*mX; //estado 1
228     X23 = analogRead(X2_3)*mX; //estado 2
229     Y3 = X23; //salida
230     //Ley de control u = -Kx
231     U3 = Kp_3*R - K1_3*X13 - K2_3*X23;
232
233     U3 = constrain(U3,0,3.3); //saturación
234 }
235
236 // ===== Control LQI =====
237 void control_4(){
238     X14 = analogRead(X1_4)*mX; //estado 1
239     X24 = analogRead(X2_4)*mX; //estado 2
240     Y4 = X24; //salida
241

```



```

242 X0_4 += Tseg*(R - Y4); //estado 0
243
244 //Ley de control u = -Kx
245 U4 = K0_4*X0_4 - K1_4*X14 - K2_4*X24;
246
247 U4 = constrain(U4,0,3.3); //saturación
248 }
249
250 //Muestreo uniforme
251 void espera(){
252     TS_code = millis()- TIC;
253     TC = TS - TS_code;
254     if (TS_code < TS) delay(TC);
255     TIC = millis();
256 }
257
258 //Comunicación con serial monitor y serial plotter
259 void coms_arduino_ide(){
260
261     Serial.print("R:"); Serial.print(R); //referencia
262
263     Serial.print(" | Y1:"); Serial.print(Y1); Serial.print(" U1:"); Serial.print(
U1); //sistema 1
264     Serial.print(" | Y2:"); Serial.print(Y2); Serial.print(" U2:"); Serial.print(
U2); //sistema 2
265     Serial.print(" | Y3:"); Serial.print(Y3); Serial.print(" U3:"); Serial.print(
U3); //sistema 3
266     Serial.print(" | Y4:"); Serial.print(Y4); Serial.print(" U4:"); Serial.println
(U4); //sistema 4
267 }
268
269 //Comunicación con monitor de pyhton
270 void coms_python(float* Rp,
271     float* X11,float* Y1p, float* U1p,
272     float* X12,float* Y2p, float* U2p,
273     float* X13,float* Y3p, float* U3p,
274     float* X14,float* Y4p, float* U4p)
275 {
276     float t = TIC/1000.0;
277
278     byte* tB = (byte*)&t; //tiempo
279     byte* RB = (byte*)&Rp; //referencia
280
281     byte* X1B1 = (byte*)&X11; //estado 1 sistema 1 AdP
282     byte* Y1B = (byte*)&Y1p; //salida sistema 1 AdP
283     byte* U1B = (byte*)&U1p; //control sistema 1 AdP
284
285     byte* X1B2 = (byte*)&X12; //estado 1 sistema 2 Integral AdP

```

```

286 byte* Y2B = (byte*)(Y2p); //salida sistema 2 Integral AdP
287 byte* U2B = (byte*)(U2p); //control sistema 2 Integral AdP
288
289 byte* X1B3 = (byte*)(X13); //estado 1 sistema 3 LQR
290 byte* Y3B = (byte*)(Y3p); //salida sistema 3 LQR
291 byte* U3B = (byte*)(U3p); //control sistema 3 LQR
292
293 byte* X1B4 = (byte*)(X14); //estado 1 sistema 4 LQR
294 byte* Y4B = (byte*)(Y4p); //salida sistema 4 LQR
295 byte* U4B = (byte*)(U4p); //control sistema 4 LQR
296
297 byte buf[56] = {
298     tB[0],tB[1],tB[2],tB[3], //tiempo
299     RB[0],RB[1],RB[2],RB[3], //Referencia
300
301     X1B1[0],X1B1[1],X1B1[2],X1B1[3], //estado 1 sistema 1 AdP
302     Y1B[0],Y1B[1],Y1B[2],Y1B[3], //salida sistema 1 AdP
303     U1B[0],U1B[1],U1B[2],U1B[3], //control sistema 1 AdP
304
305     X1B2[0],X1B2[1],X1B2[2],X1B2[3], //estado 1 sistema 2 Integral AdP
306     Y2B[0],Y2B[1],Y2B[2],Y2B[3], //salida sistema 2 Integral AdP
307     U2B[0],U2B[1],U2B[2],U2B[3], //control sistema 2 Integral AdP
308
309     X1B3[0],X1B3[1],X1B3[2],X1B3[3], //estado 1 sistema 3 LQR
310     Y3B[0],Y3B[1],Y3B[2],Y3B[3], //salida sistema 3 LQR
311     U3B[0],U3B[1],U3B[2],U3B[3], //control sistema 3 LQR
312
313     X1B4[0],X1B4[1],X1B4[2],X1B4[3], //estado 1 sistema 4 LQR
314     Y4B[0],Y4B[1],Y4B[2],Y4B[3], //salida sistema 4 LQR
315     U4B[0],U4B[1],U4B[2],U4B[3] //control sistema 4 LQR
316 };
317
318 uint8_t header[2] = {0xAA, 0x55};
319 Serial.write(header, 2);
320 Serial.write(buf, 56);
321 }

```

A.5. Monitor de adquisición y evaluación experimental

```

1 #!/usr/bin/env python
2
3 import sys
4 import datetime
5 import os
6 from threading import Thread
7 import serial

```

```
8 import time
9 import collections
10 import struct
11 import copy
12 import pandas as pd
13 import matplotlib.pyplot as plt
14 import platform
15 import subprocess
16
17 from PySide6.QtWidgets import (
18     QApplication,
19     QMainWindow,
20     QWidget,
21     QGridLayout,
22     QLabel,
23     QPushButton,
24     QLineEdit,
25     QVBoxLayout,
26     QHBoxLayout,
27     QCheckBox,
28     QMessageBox
29 )
30 from PySide6.QtCore import QTimer, Qt
31 import pyqtgraph as pg
32
33 # ===== CONFIG =====
34 def load_config():
35
36     if getattr(sys, 'frozen', False):
37         base_dir = os.path.dirname(sys.executable)
38     else:
39         base_dir = os.path.dirname(os.path.abspath(__file__))
40
41     config_dir = os.path.join(base_dir, "config")
42     config_path = os.path.join(config_dir, "config.ini")
43
44     if not os.path.exists(config_path):
45         return None
46
47     try:
48         df = pd.read_csv(config_path)
49
50         config = {
51             "port": df["port"][0],
52             "baud": int(df["baud"][0]),
53             "plotLength": int(df["plotLength"][0]),
54             "interval": int(df["interval"][0]),
55             "ymin": float(df["ymin"][0]),
```

```
56         "ymax": float(df["ymax"][0]),
57         "saveCSV": str(df["saveCSV"][0]).lower() in ["1", "true", "yes"]
58     }
59
60     QMessageBox.information(
61         None,
62         "Configuración",
63         "Configuración cargada desde config.ini"
64     )
65
66     return config
67
68 except Exception as e:
69
70     QMessageBox.critical(
71         None,
72         "Error",
73         f"Error leyendo config.ini:\n\n{e}"
74     )
75
76     return None
77
78 def save_config(port, baud, plotLength, interval, ymin, ymax, saveCSV):
79
80     if getattr(sys, 'frozen', False):
81         base_dir = os.path.dirname(sys.executable)
82     else:
83         base_dir = os.path.dirname(os.path.abspath(__file__))
84
85     config_dir = os.path.join(base_dir, "config")
86     os.makedirs(config_dir, exist_ok=True)
87
88     config_path = os.path.join(config_dir, "config.ini")
89
90     df = pd.DataFrame([
91         {
92             "port": port,
93             "baud": baud,
94             "plotLength": plotLength,
95             "interval": interval,
96             "ymin": ymin,
97             "ymax": ymax,
98             "saveCSV": saveCSV
99         }
100     ])
101
102     df.to_csv(config_path, index=False)
103
104     QMessageBox.information(
105         None,
```

```
104         "Configuración guardada",
105         f"Configuración guardada en:\n\n{config_path}"
106     )
107
108 def clear():
109     if os.name == "posix":
110         os.system ("clear")
111     elif os.name in ("ce", "nt", "dos"):
112         os.system ("cls")
113
114 # ===== SERIAL =====
115 class serialPlot:
116     def __init__(self, port, baud, plotLength):
117
118         self.plotMaxLength = plotLength
119
120         self.rawData = bytearray(56) # NUEVO
121
122         self.buffer = bytearray()
123
124         # [R,X1,Y,U] x 4 = 16 buffers
125         self.data = [collections.deque([0]*plotLength, maxlen=plotLength) for _ in
126 range(16)]
127
128         self.packetQueue = collections.deque()
129         self.csvPackets = []
130
131         self.isRun = True
132         self.thread = None
133
134         self.serialConnection = serial.Serial(port, baud, timeout=4)
135
136     def readSerialStart(self):
137         self.thread = Thread(target=self.backgroundThread)
138         self.thread.daemon = True
139         self.thread.start()
140
141     def backgroundThread(self):
142         time.sleep(1)
143         self.serialConnection.reset_input_buffer()
144
145         while self.isRun:
146
147             try:
148
149                 data = self.serialConnection.read(
150                     self.serialConnection.in_waiting or 1
151                 )
```

```
151         if data:
152             self.buffer.extend(data)
153
154     except serial.SerialException as e:
155
156         print("\nError serial:")
157         print(e)
158
159         self.isRun = False
160         break
161
162     #sincronización de paquetes
163     while True:
164         start = self.buffer.find(b'\xAA\x55')
165
166         if start == -1:
167             self.buffer = self.buffer[-2:]
168             break
169
170         if len(self.buffer) < start + 2 + 56:
171             break # esperar más datos
172
173         packet = self.buffer[start+2:start+2+56]
174         self.buffer = self.buffer[start+2+56:]
175
176         self.packetQueue.append(packet)
177         self.csvPackets.append(packet)
178
179     def close(self, saveCSV):
180         self.isRun = False
181         if self.thread is not None:
182             self.thread.join(timeout=1)
183
184         if self.serialConnection.is_open:
185             self.serialConnection.close()
186
187         if saveCSV:
188             self.saveCSV()
189
190     def saveCSV(self):
191
192         if not self.csvPackets:
193
194             QMessageBox.warning(
195                 None,
196                 "Sin datos",
197                 "No hay datos para guardar."
```

```
199         )
200
201         return
202
203     data = []
204
205     for packet in self.csvPackets:
206         values = []
207         for i in range(14):
208             value, = struct.unpack('f', packet[i*4:(i+1)*4])
209             values.append(value)
210
211         data.append(values)
212
213     df = pd.DataFrame(data, columns=[
214         't', 'R',
215         'X1_1', 'Y1', 'U1',
216         'X1_2', 'Y2', 'U2',
217         'X1_3', 'Y3', 'U3',
218         'X1_4', 'Y4', 'U4'
219     ])
220
221     # Ruta base compatible con .py y .exe
222     if getattr(sys, 'frozen', False):
223         base_dir = os.path.dirname(sys.executable)
224     else:
225         base_dir = os.path.dirname(os.path.abspath(__file__))
226
227     # Carpeta /data
228     data_dir = os.path.join(base_dir, "data")
229     os.makedirs(data_dir, exist_ok=True)
230
231     filename = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
232
233     # Ruta completa del CSV
234     path = os.path.join(data_dir, f"datos_{filename}.csv")
235
236     df.to_csv(path, index=False)
237
238     QMessageBox.information(
239         None,
240         "CSV guardado",
241         f"CSV guardado en:\n\n{path}"
242     )
243
244     resultados, nombres = self.calcular_J(df)
245     self.generar_reporte(df, resultados, nombres, filename, path)
246
```

```
247 def calcular_J(self, df):
248
249     dt = df['t'].diff().fillna(0)
250
251     resultados = []
252
253     for i in range(1,5):
254         x1 = df[f'X1_{i}']
255         x2 = df[f'Y_{i}']
256         u = df[f'U_{i}']
257
258         J = ((x1**2 + 5*x2**2 + u**2) * dt).sum()
259         resultados.append(J)
260
261     nombres = ["AdP", "Int-AdP", "LQR", "LQI"]
262
263     return resultados, nombres
264
265 def generar_reporte(self, df, resultados, nombres, filename, path):
266
267     t = df['t']
268     R = df['R']
269
270     fig, axs = plt.subplots(2, 2, figsize=(12, 8))
271     axs = axs.flatten()
272
273     for i in range(4):
274         ax = axs[i]
275
276         Y = df[f'Y_{i+1}']
277         U = df[f'U_{i+1}']
278
279         ax.plot(t, R, 'r', label='Referencia')
280         ax.plot(t, Y, 'k', label='Salida')
281         ax.plot(t, U, 'b--', label='Control')
282
283         ax.set_title(nombres[i])
284         ax.grid()
285         ax.legend()
286
287     # Mejor índice
288     best = resultados.index(min(resultados))
289
290     # Título general
291     fig.suptitle(filename, fontsize=14)
292
293     # === NUEVO BLOQUE DE TEXTO ===
294
```



```

295     # Título del índice
296     fig.text(0.5, 0.14, "Índice de desempeño", ha='center', fontsize=12,
weight='bold')
297
298     # Fórmula (LaTeX)
299     fig.text(0.5, 0.11, r"$J = \int (x^T Q x + u^T R u)\, dt$", ha='center',
fontsize=11)
300
301     # Nombres y valores alineados por columnas
302     for i in range(4):
303         fig.text(0.2 + i*0.2, 0.08, nombres[i], ha='center', fontsize=10)
304         fig.text(0.2 + i*0.2, 0.05, f"J = {resultados[i]:.2f}", ha='center',
fontsize=10)
305
306     # Mejor controlador
307     mejor_texto = f"MEJOR: {nombres[best]}"
308     fig.text(0.5, 0.02, mejor_texto, ha='center', fontsize=12, weight='bold')
309
310     plt.tight_layout(rect=[0, 0.16, 1, 0.95])
311
312     # Guardar imagen
313     img_path = path.replace(".csv", ".png")
314     plt.savefig(img_path, dpi=300)
315
316     manager = plt.get_current_fig_manager()
317
318     try:
319         manager.window.showMaximized()
320     except:
321         pass
322
323     plt.show(block=False)
324     plt.pause(0.1)
325
326     QMessageBox.information(
327         None,
328         "Reporte generado",
329         f"Imagen guardada en:\n\n{img_path}"
330     )
331
332     # ===== SETUP WINDOW =====
333     class SetupWindow(QWidget):
334
335         def __init__(self):
336             super().__init__()
337
338             self.setWindowTitle("Monitor Serial v2.1 - FIME UANL")
339             self.setMinimumWidth(350)

```

```
340 layout = QVBoxLayout()
341
342
343 title = QLabel(
344     "Monitor de Puerto Serial\n"
345     "v2.1 - 7 de mayo del 2026\n"
346     "Departamento de Electrónica y Automatización\n"
347     "FIME - UANL"
348 )
349
350 title.setStyleSheet("""
351     font-size: 14pt;
352     font-weight: bold;
353 """)
354
355 title.setAlignment(Qt.AlignCenter)
356
357 layout.addWidget(title)
358
359 # ===== Inputs =====
360
361 self.portEdit = QLineEdit("COM4")
362 self.baudEdit = QLineEdit("115200")
363 self.samplesEdit = QLineEdit("50")
364 self.intervalEdit = QLineEdit("50")
365 self.yminEdit = QLineEdit("-0.1")
366 self.ymaxEdit = QLineEdit("3.3")
367
368 layout.addWidget(QLabel("Puerto"))
369 layout.addWidget(self.portEdit)
370
371 layout.addWidget(QLabel("Baudrate"))
372 layout.addWidget(self.baudEdit)
373
374 layout.addWidget(QLabel("Samples"))
375 layout.addWidget(self.samplesEdit)
376
377 layout.addWidget(QLabel("Intervalo (ms)"))
378 layout.addWidget(self.intervalEdit)
379
380 layout.addWidget(QLabel("Y min"))
381 layout.addWidget(self.yminEdit)
382
383 layout.addWidget(QLabel("Y max"))
384 layout.addWidget(self.ymaxEdit)
385
386 # ===== Checkboxes =====
387
```

```
388     self.saveCSVCheck = QCheckBox("Guardar CSV")
389     self.saveConfigCheck = QCheckBox("Guardar configuración")
390     self.useConfigCheck = QCheckBox("Usar configuración guardada")
391
392     layout.addWidget(self.saveCSVCheck)
393     layout.addWidget(self.saveConfigCheck)
394     layout.addWidget(self.useConfigCheck)
395
396     # ===== Botón =====
397
398     self.startButton = QPushButton("INICIAR")
399     self.startButton.clicked.connect(self.startMonitor)
400
401     layout.addWidget(self.startButton)
402
403     self.setLayout(layout)
404
405     # ===== Cargar config automáticamente =====
406
407     self.config = load_config()
408
409     if self.config:
410         self.useConfigCheck.setChecked(True)
411         self.saveCSVCheck.setChecked(self.config["saveCSV"])
412         self.portEdit.setText(str(self.config["port"]))
413         self.baudEdit.setText(str(self.config["baud"]))
414         self.samplesEdit.setText(str(self.config["plotLength"]))
415         self.intervalEdit.setText(str(self.config["interval"]))
416         self.yminEdit.setText(str(self.config["ymin"]))
417         self.ymaxEdit.setText(str(self.config["ymax"]))
418
419     def startMonitor(self):
420
421         try:
422
423             # ===== Config guardada =====
424
425             if self.useConfigCheck.isChecked() and self.config:
426
427                 port = self.config["port"]
428                 baud = self.config["baud"]
429                 plotLength = self.config["plotLength"]
430                 interval = self.config["interval"]
431                 ymin = self.config["ymin"]
432                 ymax = self.config["ymax"]
433
434             else:
```

```
436         port = self.portEdit.text()
437         baud = int(self.baudEdit.text())
438         plotLength = int(self.samplesEdit.text())
439         interval = int(self.intervalEdit.text())
440         ymin = float(self.yminEdit.text())
441         ymax = float(self.ymaxEdit.text())
442
443     saveCSV = self.saveCSVCheck.isChecked()
444
445     # ===== Guardar config =====
446
447     if self.saveConfigCheck.isChecked():
448
449         save_config(
450             port,
451             baud,
452             plotLength,
453             interval,
454             ymin,
455             ymax,
456             saveCSV
457         )
458
459     # ===== Serial =====
460
461     self.serialObj = serialPlot(
462         port,
463         baud,
464         plotLength
465     )
466
467     self.serialObj.readSerialStart()
468
469     # ===== Monitor =====
470
471     self.monitor = Monitor(
472         self.serialObj,
473         interval,
474         ymin,
475         ymax,
476         saveCSV
477     )
478
479     self.monitor.showMaximized()
480
481     self.hide()
482
483     # guardar para closeEvent
```

```
484         self.saveCSV = saveCSV
485
486     except Exception as e:
487
488         QMessageBox.critical(
489             self,
490             "Error",
491             str(e)
492         )
493
494 # ===== GUI =====
495 class Monitor(QMainWindow):
496     def __init__(self, serialObj, interval, ymin, ymax, saveCSV):
497
498         super().__init__()
499
500         self.setWindowTitle("Monitor de Control")
501
502         self.s = serialObj
503
504         central = QWidget()
505         self.setCentralWidget(central)
506
507         layout = QGridLayout()
508         central.setLayout(layout)
509
510         self.plots = []
511         self.curves = []
512
513         self.names = ["Control por Asignación de Polos", "Control Integral por
514 Asignación de Polos", "Control LQR", "Control LQI"]
515         for i in range(4):
516             p = pg.PlotWidget(title=self.names[i])
517             p.setYRange(ymin, ymax)
518             p.setBackground('w')
519             p.showGrid(x=True, y=True)
520             p.addLegend()
521
522             r = p.plot(
523                 pen=pg.mkPen('r', width=2),
524                 name='Referencia'
525             )
526
527             y = p.plot(
528                 pen=pg.mkPen('k', width=2),
529                 name='Salida'
530             )
```

```

531         u = p.plot(
532             pen=pg.mkPen(color='b', width=2, style=Qt.DashLine),
533             name='Control'
534         )
535
536         self.plots.append(p)
537         self.curves.append((r,y,u))
538
539         layout.addWidget(p, i//2, i%2)
540
541         self.timer = QTimer()
542         self.timer.timeout.connect(self.update)
543         self.timer.start(interval)
544         self.saveCSV = saveCSV
545
546     def update(self):
547
548         if len(self.s.packetQueue) == 0:
549             return
550
551         packet = self.s.packetQueue[-1]
552         self.s.packetQueue.clear()
553
554         values = [struct.unpack('f', packet[i*4:(i+1)*4])[0] for i in range(14)]
555
556         R = values[1]
557
558         idx = 2
559
560         for i in range(4):
561             X1 = values[idx]
562             Y = values[idx+1]
563             U = values[idx+2]
564             idx += 3
565
566             self.s.data[i*4].append(R)
567             self.s.data[i*4+1].append(X1)
568             self.s.data[i*4+2].append(Y)
569             self.s.data[i*4+3].append(U)
570
571             r,y,u = self.curves[i]
572
573             r.setData(list(self.s.data[i*4]))
574             y.setData(list(self.s.data[i*4+2]))
575             u.setData(list(self.s.data[i*4+3]))
576
577     def closeEvent(self, event):
578

```

```
579         try:
580             self.s.close(self.saveCSV)
581         except:
582             pass
583
584         event.accept()
585
586     # ===== MAIN =====
587 def main():
588
589     app = QApplication(sys.argv)
590
591     setup = SetupWindow()
592     setup.show()
593
594     sys.exit(app.exec())
595
596
597 if __name__ == "__main__":
598     main()
```

B. Controladores

Parte del desarrollo e implementación experimental de los controladores presentados en esta sección se realizó tomando como referencia las prácticas, ejemplos y metodologías disponibles en el repositorio [4]. Particularmente, dicho material sirvió como apoyo para la implementación de estrategias de asignación de polos, control integral, observadores de estado y estructuras de programación utilizadas en el sistema experimental.

B.1. Control por asignación de polos

El diseño mediante asignación de polos consiste en determinar una ley de control por retroalimentación de estados que permita ubicar los polos del sistema en posiciones previamente especificadas del plano complejo.

El diseño de los controladores mediante asignación de polos se realizó utilizando técnicas clásicas de retroalimentación de estados descritas en [1, 3].

La ley de control propuesta es:

$$u = -Kx + gr \quad (15)$$

con:

$$K = \begin{bmatrix} k_1 & k_2 \end{bmatrix} \quad (16)$$

Por lo tanto, la dinámica en lazo cerrado queda definida por:

$$\dot{x} = (A - BK)x + Bgr \quad (17)$$

La matriz del sistema en lazo cerrado es:

$$A - BK = \begin{bmatrix} -\frac{1}{R_1 C_1} - \frac{1}{R_2 C_1} - \frac{k_1}{R_1 C_1} & \frac{1}{R_2 C_1} - \frac{k_2}{R_1 C_1} \\ \frac{1}{R_2 C_2} & -\frac{1}{R_2 C_2} - \frac{1}{R_3 C_2} \end{bmatrix} \quad (18)$$

El polinomio característico del sistema se obtiene mediante:

$$|\lambda I - (A - BK)| = 0 \quad (19)$$

Desarrollando el determinante:

$$\begin{aligned} |\lambda I - (A - BK)| = & \lambda^2 + \left(\frac{1}{R_1 C_1} + \frac{1}{R_2 C_1} + \frac{1}{R_2 C_2} + \frac{1}{R_3 C_2} + \frac{k_1}{R_1 C_1} \right) \lambda \\ & + \frac{1}{R_1 R_2 C_1 C_2} + \frac{1}{R_1 R_3 C_1 C_2} + \frac{1}{R_2 R_3 C_1 C_2} \\ & + \frac{k_1}{R_1 R_2 C_1 C_2} + \frac{k_1}{R_1 R_3 C_1 C_2} + \frac{k_2}{R_1 R_2 C_1 C_2} \end{aligned} \quad (20)$$

Se selecciona un polinomio deseado de segundo orden:

$$(\lambda - a_1)(\lambda - a_2) = \lambda^2 + \alpha_1 \lambda + \alpha_2 \quad (21)$$

con:

$$\alpha_1 = -(a_1 + a_2) \quad (22)$$

$$\alpha_2 = a_1 a_2 \quad (23)$$

Igualando coeficientes del polinomio característico, se obtienen las ganancias del controlador:

$$k_1 = R_1 C_1 \left(\alpha_1 - \frac{1}{R_1 C_1} - \frac{1}{R_2 C_1} - \frac{1}{R_2 C_2} - \frac{1}{R_3 C_2} \right) \quad (24)$$

$$k_2 = R_1 R_2 C_1 C_2 \left(\alpha_2 - \frac{1}{R_1 R_2 C_1 C_2} - \frac{1}{R_1 R_3 C_1 C_2} - \frac{1}{R_2 R_3 C_1 C_2} - \frac{k_1}{R_1 R_2 C_1 C_2} - \frac{k_1}{R_1 R_3 C_1 C_2} \right) \quad (25)$$

Para la implementación experimental se seleccionaron polos complejos conjugados con el objetivo de obtener una respuesta rápida y amortiguada.

B.2. Control integral por asignación de polos

Con el propósito de eliminar el error en estado estacionario ante referencias constantes, se incorporó una acción integral al sistema. Para ello se define un nuevo estado integrador:

$$\dot{x}_0 = \int (r - y) dt \quad (26)$$

Por lo tanto:

$$\dot{x}_0 = r - y \quad (27)$$

La ley de control se define como:

$$u = k_0 x_0 - k_1 x_1 - k_2 x_2 \quad (28)$$

Definiendo el vector de estado aumentado:

$$X_a = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} \quad (29)$$

el sistema aumentado queda expresado como:

$$\dot{X}_a = A_{cl} X_a + B_a r \quad (30)$$

con:

$$A_{cl} = \begin{bmatrix} 0 & 0 & -1 \\ \frac{k_0}{R_1 C_1} & -\frac{1}{R_1 C_1} - \frac{1}{R_2 C_1} - \frac{k_1}{R_1 C_1} & \frac{1}{R_2 C_1} - \frac{k_2}{R_1 C_1} \\ 0 & \frac{1}{R_2 C_2} & -\frac{1}{R_2 C_2} - \frac{1}{R_3 C_2} \end{bmatrix} \quad (31)$$

El polinomio característico del sistema aumentado se obtiene mediante:

$$|\lambda I - A_{cl}| = 0 \quad (32)$$

Desarrollando el determinante:

$$\begin{aligned} |\lambda I - A_{cl}| = & \lambda^3 + \left(\frac{1}{R_1 C_1} + \frac{1}{R_2 C_1} + \frac{1}{R_2 C_2} + \frac{1}{R_3 C_2} + \frac{k_1}{R_1 C_1} \right) \lambda^2 \\ & + \left(\frac{1}{R_1 R_2 C_1 C_2} + \frac{1}{R_1 R_3 C_1 C_2} + \frac{1}{R_2 R_3 C_1 C_2} \right. \\ & + \frac{k_1}{R_1 R_2 C_1 C_2} + \frac{k_1}{R_1 R_3 C_1 C_2} + \frac{k_2}{R_1 R_2 C_1 C_2} \Bigg) \lambda \\ & + \frac{k_0}{R_1 R_2 C_1 C_2} \end{aligned} \quad (33)$$

Se define un polinomio deseado de tercer orden:

$$(\lambda - a_1)(\lambda - a_2)(\lambda - a_3) = \lambda^3 + \alpha_1 \lambda^2 + \alpha_2 \lambda + \alpha_3 \quad (34)$$

con:

$$\alpha_1 = -(a_1 + a_2 + a_3) \quad (35)$$

$$\alpha_2 = a_1 a_2 + a_1 a_3 + a_2 a_3 \quad (36)$$

$$\alpha_3 = -a_1 a_2 a_3 \quad (37)$$

Igualando coeficientes del polinomio característico, se obtienen las ganancias:

$$k_0 = R_1 R_2 C_1 C_2 \alpha_3 \quad (38)$$

$$k_1 = R_1 C_1 \left(\alpha_1 - \frac{1}{R_1 C_1} - \frac{1}{R_2 C_1} - \frac{1}{R_2 C_2} - \frac{1}{R_3 C_2} \right) \quad (39)$$

$$k_2 = R_1 R_2 C_1 C_2 \left(\alpha_2 - \frac{1}{R_2 R_3 C_1 C_2} - \frac{1 + k_1}{R_1 R_3 C_1 C_2} \right) - 1 - k_1 \quad (40)$$

La incorporación del integrador permite que el sistema se comporte como un sistema tipo 1 respecto a referencias constantes, eliminando el error en estado estacionario.

B.3. Observador de Luenberger

Para los controladores basados en asignación de polos se implementó un observador de Luenberger, debido a que experimentalmente únicamente se mide el voltaje correspondiente al segundo capacitor.

El observador de estados implementado corresponde a un observador de orden completo basado en el principio de separación, desarrollado en [1, 3].

El observador se define mediante:

$$\dot{\hat{x}} = A\hat{x} + Bu + H(y - C\hat{x}) \quad (41)$$

con:

$$H = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} \quad (42)$$

La dinámica del error de estimación queda dada por:

$$\dot{e} = (A - HC)e \quad (43)$$

La matriz del observador es:

$$A - HC = \begin{bmatrix} -\frac{1}{R_1C_1} - \frac{1}{R_2C_1} & \frac{1}{R_2C_1} - h_1 \\ \frac{1}{R_2C_2} & -\frac{1}{R_2C_2} - \frac{1}{R_3C_2} - h_2 \end{bmatrix} \quad (44)$$

El polinomio característico del observador se obtiene mediante:

$$|\lambda I - (A - HC)| = 0 \quad (45)$$

Desarrollando el determinante:

$$\begin{aligned} |\lambda I - (A - HC)| = & \lambda^2 + \left(\frac{1}{R_1C_1} + \frac{1}{R_2C_1} + \frac{1}{R_2C_2} + \frac{1}{R_3C_2} + h_2 \right) \lambda \\ & + \frac{1}{R_1R_2C_1C_2} + \frac{1}{R_1R_3C_1C_2} + \frac{1}{R_2R_3C_1C_2} \\ & + \frac{h_2}{R_1C_1} + \frac{h_2}{R_2C_1} + \frac{h_1}{R_2C_2} \end{aligned} \quad (46)$$

Se selecciona un polinomio deseado:

$$(\lambda - b_1)(\lambda - b_2) = \lambda^2 + \beta_1\lambda + \beta_2 \quad (47)$$

con:

$$\beta_1 = -(b_1 + b_2) \quad (48)$$

$$\beta_2 = b_1 b_2 \quad (49)$$

Igualando coeficientes se obtienen las ganancias del observador:

$$h_2 = \beta_1 - \frac{1}{R_1 C_1} - \frac{1}{R_2 C_1} - \frac{1}{R_2 C_2} - \frac{1}{R_3 C_2} \quad (50)$$

$$h_1 = R_2 C_2 \left(\beta_2 - \frac{1}{R_1 R_2 C_1 C_2} - \frac{1}{R_1 R_3 C_1 C_2} - \frac{1}{R_2 R_3 C_1 C_2} - \frac{h_2}{R_1 C_1} - \frac{h_2}{R_2 C_1} \right) \quad (51)$$

Los polos del observador se seleccionaron considerablemente más rápidos que los polos del controlador, con el propósito de garantizar una convergencia rápida del error de estimación.